UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.

HR EXCELLENCE IN RESEARCH

04: Emerging Themes

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Emerging Themes

Future networks are evolving from passive infrastructures into intelligent systems able to adapt to users, services, and context.

- **From manual operation to autonomy**
  Network management is moving from human-written scripts and manual configuration toward systems that can monitor, decide, and react automatically.
- **From software-defined to AI-managed networks**
  AI will not simply be an external optimization tool. It will become part of the network control logic, supporting decisions across radio, transport, core, edge, and cloud domains.
- **From risky live changes to digital experimentation**
  Network Digital Twins will provide virtual replicas where operators can test new configurations, train AI models, and predict failures before acting on the real infrastructure.
- **From bit transport to meaning-aware communication**
  Semantic Communications will move the focus from transmitting every bit correctly to transmitting the information that is actually relevant for the application or task.

AI-Driven Network Automation

Why Manual Management is No Longer Enough

Future networks are too complex and dynamic for purely manual control.

Complexity beyond human scale

- Many parameters must be optimized together: radio, slicing, edge, cloud, security.
- The number of possible configurations becomes too large for manual management.

Operational cost and scalability

- Manual configuration and troubleshooting increase OPEX.
- New services must be deployed faster than traditional processes allow.

Need for fast reactions

- Failures, congestion, mobility, and energy conditions change continuously.
- Some applications require decisions in milliseconds, not after human intervention.

From Assisted Operation to Autonomous Networks

Network autonomy is not a single step, but a gradual evolution from human-controlled operations to self-managed systems.

Levels of autonomy

- **L0–L1: Human-driven operations**
  The network may provide tools or recommendations, but humans still execute most actions.
- **L2–L3: Partial autonomy**
  Some closed loops are automated in specific domains, such as RAN optimization, core management, or fault detection.
- **L4–L5: High / full autonomy**
  The network coordinates decisions across domains and manages services with minimal or no human intervention.

Where we are today

Most real deployments are still between **L2 and L3**: automation exists, but it is often limited to specific domains or predefined scenarios.

Main barriers to higher autonomy

- Cross-domain coordination: RAN, core, transport, and cloud must work together.
- Trust and explainability: operators need to understand AI decisions.
- Legacy systems: not all devices expose programmable interfaces or rich telemetry.
- Security and liability: autonomous decisions may have operational consequences.

Intent-Based Networking (IBN) Concept: The Shift to Declarative Management

Intent-Based Networking changes the way networks are managed: the operator specifies the desired outcome, while the network decides how to realize it.

From commands to goals

- **Traditional approach:** configure devices step by step.
- **Intent-based approach:** describe the service objective.
- The system translates the intent into policies, resources, and configurations.

Example

Instead of:

"Configure VLANs, routing rules, and ACLs manually."

The operator says:

"Keep guest traffic isolated and guarantee low latency for critical services."

Main functions of IBN

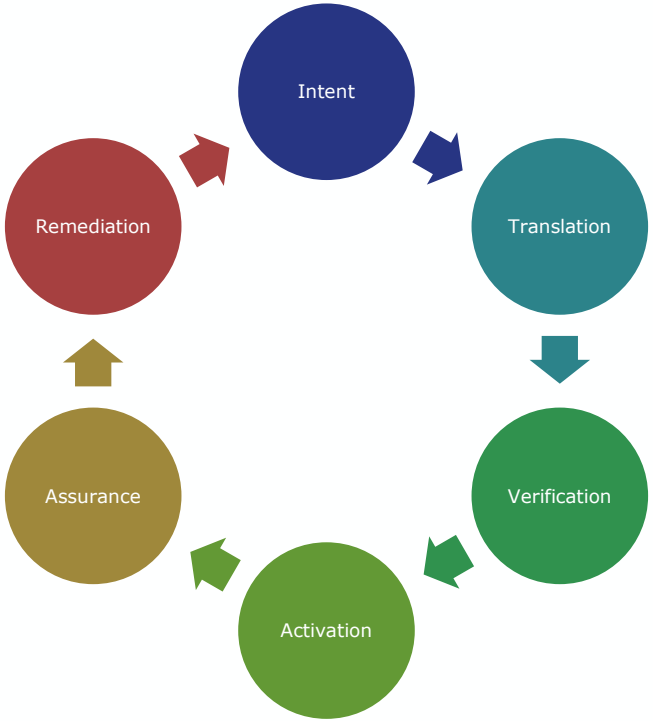
- **Translation:** convert high-level intent into technical requirements.
- **Activation:** apply the required configuration across the infrastructure.
- **Assurance:** continuously check that the intent is still satisfied.
- **Optimization:** adapt the network when conditions change.

The IBN Closed-Loop Architecture: The Lifecycle of Intent Execution

An intent-based network does not only apply a configuration once. It continuously checks whether the intended goal is still satisfied.

Main steps

1. **Ingestion**
   Capture the user or service intent.
2. **Translation**
   Convert the intent into technical requirements and policies.
3. **Verification**
   Check that the planned configuration is safe and feasible.
4. **Activation**
   Apply the configuration across the network.
5. **Assurance**
   Monitor the network and detect deviations from the intent.



Step 1: Intent Ingestion

The first step is to transform a human request into a clear and measurable network objective.

From natural language to intent

- Operators may express goals in plain language.
- NLP/LLMs can help interpret the request.
- The system must convert vague goals into precise service requirements.

What must be extracted

- Target users, applications, or services.
- Required latency, reliability, bandwidth, or coverage.
- Time, location, and priority constraints.

Why it is difficult

- Human requests are often incomplete or ambiguous.
- The system may need to ask clarifying questions.
- The intent must be linked to real network entities and resources.

Step 2: Intent Translation

After ingestion, the network must transform the intent into technical rules and check whether it is compatible with existing policies.

From intent to technical constraints

- High-level goals must become measurable requirements.
- Examples: latency, bandwidth, reliability, isolation, security.
- The system maps these requirements to network resources and policies
  - **YANG (Yet Another Next Generation)** to describe the desired state.

Why conflicts appear

- Different users or services may request incompatible objectives.
- The same resources may be needed by multiple slices or applications.
- Business goals may compete: lowest cost, lowest latency, highest reliability, lowest energy.

How the system reacts

- Detect the conflict before activation.
- Apply priority rules when needed.
- Suggest alternatives or negotiate a feasible solution.

Step 3: Intent Verification

Before an autonomous system changes the network, it must verify that the action is safe, feasible, and compliant with existing rules.

What verification checks

- The service remains reachable.
- Security and isolation policies are respected.
- The change does not create routing loops or instability.
- Performance requirements can still be satisfied.

Why it matters

- Autonomous actions may affect many users and services.
- A wrong configuration can propagate quickly across domains.
- Verification acts as a safety barrier before activation.

Possible techniques

- Formal verification for strict correctness checks.
- Graph-based models to reason on topology and reachability.
- Digital twins to test actions before applying them live.

Step 4: Intent Activation

After verification, the network must safely apply the selected configuration to the real infrastructure.

From model to deployment

- The validated intent is translated into concrete network actions.
- Controllers (e.g., SDN) and orchestrators (e.g., MANO) apply changes across devices, functions, and domains.
- The system hides vendor-specific configuration details from the operator.

Why activation must be controlled

- A partial update can create inconsistent network states.
- Changes may affect multiple domains at the same time.
- The system must be ready to stop, retry, or roll back.

Safety mechanisms

- Apply changes only after verification.
- Monitor the effect immediately after deployment.
- Roll back if the new state creates problems.

Step 5: Intent Assurance & Remediation

After activation, the network must continuously check whether the intended service goal is still being met.

From monitoring to assurance

- Monitoring tells us what is happening in the network.
- Assurance checks whether the service intent is actually satisfied.
- The system compares the **intended state** with the **observed state**.

Detecting intent drift

- Traffic, failures, mobility, and load may change over time.
- These changes can make the network deviate from the original intent.
- The system must detect this deviation automatically.

Closing the loop

- If the intent is violated, the network can trigger remediation.
- Possible actions include rerouting, scaling resources, reconfiguration, or rollback.
- The goal is to maintain the service without waiting for manual intervention.

Zero-Touch Service Management (ZSM) Vision

Zero-Touch Service Management extends automation from single network actions to the entire lifecycle of a service.

What ZSM aims to automate

- **Provisioning:** create and configure services with minimal manual intervention.
- **Operation:** monitor performance and adapt resources during service execution.
- **Optimization:** improve latency, reliability, energy usage, or cost over time.
- **Termination:** release resources when the service is no longer needed.

Why it matters

- Future services may span RAN, transport, core, edge, and cloud.
- Manual coordination across all these domains is slow and error-prone.
- ZSM aims to make service management faster, scalable, and more consistent.

ZSM is the operational vision where the network manages services end-to-end with humans mainly defining objectives and constraints.

ZSM Reference Architecture

Zero-touch management requires coordination across several domains, not only automation inside one part of the network.

Domain-level autonomy

- Each domain can manage local problems and optimizations.
- Examples: RAN, transport, core, edge/cloud.
- Local decisions are faster and reduce unnecessary central coordination.

End-to-end service management

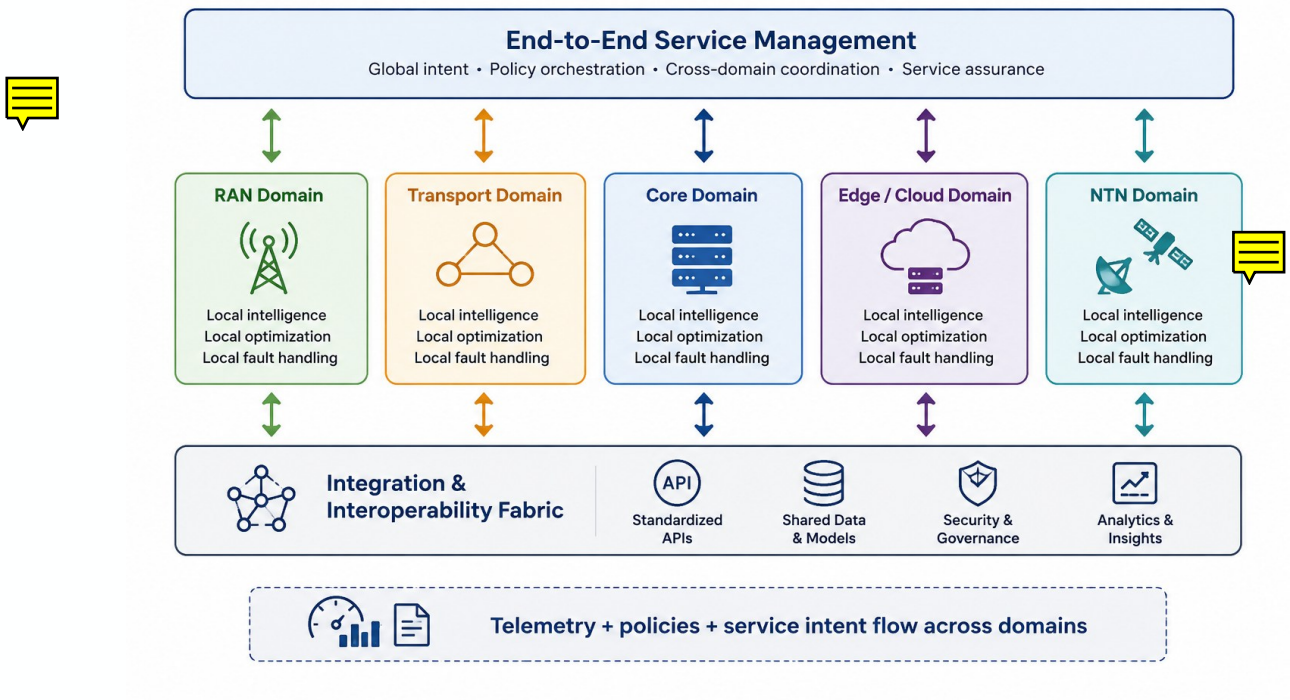
- A service often crosses multiple domains.
- The E2E layer coordinates domain decisions to satisfy the global service goal.
- This is essential for network slicing and mission-critical services.

Integration and interoperability

- Domains must exchange data, policies, and decisions.
- Standard interfaces help avoid vendor lock-in.
- The system needs a shared view of service state and performance.

ZSM is not one big controller managing everything; it is a coordinated set of domain loops working together under an end-to-end service objective.

ZSM Architecture: Coordinating Multiple Network Domains



The MAPE-K Loop in ZSM

Autonomous network management is based on a continuous control loop that observes the network, decides what to do, and applies corrective actions.

The loop

- **Monitor:** collect telemetry from the network.
- **Analyze:** detect problems, anomalies, or deviations from the desired state.
- **Plan:** choose the best action to restore or improve the service.
- **Execute:** apply the selected action through controllers or orchestrators.

The Knowledge component

- Stores past network states and previous decisions.
- Contains policies, constraints, and safety rules.
- Helps the system improve future decisions.

AI for Self-Healing (Fault Management)

AI can help networks detect problems early, understand their causes, and trigger corrective actions before users are affected.

From reactive to predictive

- Traditional fault management reacts after a failure occurs.
- AI-based systems can detect weak signals before service degradation.
- The goal is to prevent outages, not only repair them.

Understanding the root cause

- One physical problem may generate many alarms.
- AI can help distinguish symptoms from the real cause.
- This reduces troubleshooting time and unnecessary interventions.

Closing the self-healing loop

- Detect the anomaly.
- Identify the likely cause.
- Apply a corrective action.
- Check whether the action solved the problem.

AI for Self-Optimization and Sustainable Networking

AI can help future networks continuously optimize performance while reducing energy consumption.

Self-optimization

- AI can adapt radio resources, routing, beamforming, and edge/cloud resources according to traffic and mobility.
- The network learns from observed conditions and selects better configurations over time.
- The objective is to maintain good service quality without constant manual tuning.

Energy-aware operation

- Future networks cannot remain "always fully active."
- AI can predict low-traffic periods and activate sleep modes or resource consolidation.
- The goal is to move from **always on** to **always available**.

Main trade-off

- Higher performance usually requires more resources and more energy.
- Sustainable networking must balance throughput, latency, reliability, cost, and power consumption.

DRL Agents in Automation

Reinforcement Learning allows the network to learn better decisions through interaction with the environment.

Basic idea

- The network observes its current **state**.
- An AI agent selects an **action**.
- The system receives a **reward** depending on the result.
- Over time, the agent learns which actions work better.

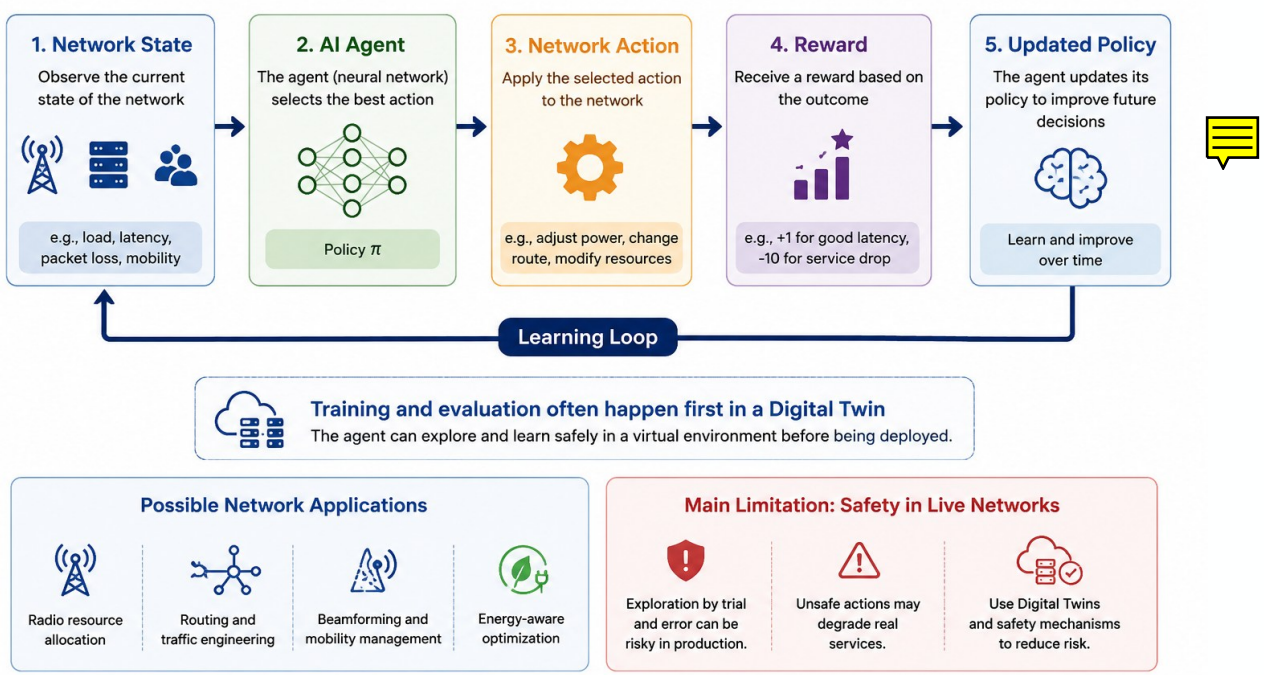
Possible network applications

- Radio resource allocation.
- Routing and traffic engineering.
- Beamforming and mobility management.
- Energy-aware optimization.

Main limitation

- Learning by trial and error is risky in a live network.
- Unsafe exploration may degrade real services.
- Digital Twins can provide a safer environment for training and testing.

Deep Reinforcement Learning for Network Control



Trustworthy Autonomy: Security and Explainability

Autonomous networks can only be adopted if their decisions are secure, understandable, and controllable.

New security risks

- AI-driven decisions may be attacked or manipulated.
- Malicious intents could be injected into the automation loop.
- Autonomous actions may propagate errors very quickly across domains.

Need for explainability

- Operators must understand why the network takes an action.
- Black-box AI makes troubleshooting and accountability difficult.
- Explanations help distinguish correct optimization from wrong behavior.

Main protection principles

- Verify intents before execution.
- Apply zero-trust principles to autonomous agents.
- Keep humans able to audit, supervise, and override decisions.

ZSM Use Case: Network Slicing

Network slicing is a good example of why zero-touch management is needed: each slice may require its own resources, policies, and service guarantees.

Why automation is needed

- Future networks may support many slices at the same time.
- Each slice can have different requirements: latency, bandwidth, reliability, security, or isolation.
- Managing the full slice lifecycle manually would be too slow and complex.

Automated slice lifecycle

- **Order:** a user or application requests a service through an intent or API.
- **Provision:** the system allocates resources across RAN, transport, core, and edge/cloud.
- **Monitor:** telemetry is used to check whether the slice meets its SLA.
- **Adapt or terminate:** the system scales, repairs, or releases resources when needed.

Main benefit

- Faster slice deployment.
- Better resource usage.
- Continuous SLA assurance.
- Reduced manual coordination across domains.

The Practical Challenge: Open and Multi-Vendor Networks

Autonomous networking cannot work well in closed, vendor-specific environments. It needs open interfaces, shared models, and cloud-native automation tools.

Why openness matters

- Future networks are multi-vendor and multi-domain.
- Proprietary interfaces make automation harder and slower.
- Open APIs and common data models allow different systems to cooperate.

Open RAN as an example

- O-RAN opens parts of the radio access network.
- It enables software-based control and automation of RAN functions.
- It allows external intelligence, such as AI agents, to optimize radio behavior.

Cloud-native automation

- Platforms such as ONAP and Nephio show two possible directions.
- ONAP focuses on service orchestration across complex networks.
- Nephio follows a Kubernetes/GitOps approach for cloud-native network functions.

Future Roadmap: Self-Evolving Networks

The long-term vision is a network that does not only execute predefined scripts, but can improve its own operational logic over time.

Evolution path

1. **Manual configuration:** humans configure devices directly.
2. **Template-based automation:** scripts execute predefined actions.
3. **AI-assisted operations:** AI suggests actions, humans approve.
4. **Self-evolving networks:** the system detects problems, proposes fixes, tests them, and applies safe updates.

Role of Generative AI

- Translate high-level intents into possible configurations or code.
- Help operators explore alternative solutions.
- Support faster troubleshooting and service design.

Safety requirement

- Generated actions must be verified before deployment.
- Digital Twins can test changes before the live network is modified.
- Human supervision remains important for critical decisions.

Network Digital Twins

What is a Digital Twin?

A Digital Twin is a dynamic virtual representation of a real object, process, or system.

Unlike a static model, it is continuously updated through data exchanged with the physical world.

Core idea

- The physical system sends data to update the virtual model.
- The virtual model is used to analyze, simulate, and predict behavior.
- Insights from the twin can guide actions on the real system.

Key capabilities

- **Synchronization:** keeping the virtual model aligned with reality.
- **Analytics:** extracting knowledge from real-time and historical data.
- **What-if simulation:** testing scenarios without risking the real system.
- **Optimization:** improving operation before applying changes physically.

"A Digital Twin is a bridge between the physical and digital worlds, ensuring that every physical change has a digital footprint."

Introduction to Network Digital Twins (NDT)

A Network Digital Twin is a dynamic virtual replica of the physical network, continuously synchronized with real network data.

What the twin reflects

- **Network state:** active nodes, failures, congestion, sleeping components.
- **Topology:** physical links, logical paths, domains, and service routes.
- **Configuration:** policies, parameters, slices, and resource allocation.
- **Traffic behavior:** flows, mobility patterns, load, and service demand.

Why it is useful

- It gives operators and AI systems a virtual view of the current network.
- It allows "what-if" analysis before applying changes.
- It supports prediction of faults, bottlenecks, and performance degradation.
- It provides a safe environment for testing autonomous decisions.

Digital Twin vs. Classical Simulation

A simulation helps us study possible scenarios; a Digital Twin follows the real system during operation.

Classical Simulation	Digital Twin
Mostly **offline**	Continuously **online**
Uses historical, static, or synthetic data	Uses real-time data from the physical system
Mainly supports planning and validation	Supports monitoring, prediction, and optimization
Describes what **could happen**	Mirrors what **is happening**
Usually one-way: model → results	Bidirectional: physical system ↔ virtual model

What a Network Digital Twin Contains

A Network Digital Twin is not only a virtual model. It combines the real network, its digital representation, data exchange, services, and intelligence.

Main components

- **Physical network**
  The real infrastructure: devices, users, links, radio environment, edge/cloud resources.
- **Virtual model**
  The software replica used to represent topology, behavior, traffic, and performance.
- **Data connection**
  The continuous exchange of telemetry, KPIs, configuration updates, and control actions.
- **Twin services**
  Functions offered by the twin, such as what-if analysis, testing, prediction, and maintenance support.
- **Intelligence layer**
  AI/ML algorithms that analyze the twin, compare virtual and physical states, and support decisions.

Synchronization: Keeping the Twin Aligned with Reality

A Network Digital Twin is useful only if it remains sufficiently synchronized with the physical network.

Why synchronization matters

- The real network changes continuously: traffic, mobility, failures, load, and configurations.
- If the twin is outdated, its predictions and decisions may become wrong.
- Autonomous actions based on a stale twin can create risks for live services.

What must be synchronized

- Network state and topology.
- Traffic and performance indicators.
- Configuration and policy changes.
- Events, faults, and anomalies across domains.

Main challenge

- The faster the real network changes, the more frequently the twin must be updated.
- Synchronization must balance accuracy, delay, data volume, and processing cost.

Modeling Fidelity & Scale

A Network Digital Twin must balance two competing needs: being accurate enough to support decisions, but lightweight enough to run at network scale.

Two modeling levels

- **Packet-level modeling**
  Very detailed, but computationally expensive and difficult to scale.
- **Flow-level modeling**
  More scalable, but less precise for detailed protocol or performance analysis.

The hybrid approach

- Use coarse models for the whole network.
- Zoom in only where more detail is needed.
- Increase fidelity when anomalies, SLA violations, or critical events appear.

Digital twin as a safe experimentation environment

A Network Digital Twin allows operators and AI agents to test decisions virtually before applying them to the live network.

What can be tested

- New routing or traffic engineering policies.
- Software updates and configuration changes.
- AI models for resource allocation or self-healing.
- Failure scenarios, congestion peaks, or cyber-attacks.

Why it is useful

- It reduces the risk of disrupting real users.
- It helps estimate the impact on latency, throughput, jitter, and reliability.
- It supports safer deployment of autonomous decisions.

Typical workflow

Propose change → Test in the twin → Evaluate impact → Deploy if safe

Network Digital Twin use cases

Network Digital Twins can be used as operational laboratories where operators and AI systems test, predict, and optimize network behavior before acting on the real infrastructure.

Main use cases

- **Predictive maintenance**
  The twin can simulate hardware degradation and detect early signs of failures, allowing operators to intervene before a service outage occurs.
- **Security sandboxing**
  Attacks, vulnerabilities, and defense strategies can be tested in the twin without exposing live users or real services to risk.
- **6G RAN optimization**
  The twin can model radio propagation, mobility, beamforming, and coverage to improve radio resource management and site planning.
- **Federated digital twins**
  Network twins can interact with other twins, such as vehicle, city, energy, or industrial twins, to support cross-domain coordination.

Limits and Risks of Network Digital Twins

Network Digital Twins are powerful, but building and operating them at real network scale is still very challenging.

Main open challenges

- **Computational cost**
  High-fidelity twins may require large computing resources, especially for metropolitan-scale or real-time scenarios.
- **Data volume and velocity**
  Continuous telemetry from RAN, core, transport, and edge domains can generate massive data streams.
- **Synchronization lag**
  If the twin is not updated fast enough, it may represent an outdated network state.
- **Privacy of user and traffic data**
  Accurate twins may require sensitive information about users, mobility, sessions, and traffic behavior.
- **Interoperability and standards**
  Multi-vendor networks need common models, APIs, and reference architectures to build coherent twins.

Semantic Communications

Semantic Communications: From Bits to Meaning

Traditional networks mainly optimize the accurate transmission of bits. Semantic Communications ask whether the received information is useful for the task.

Classical communication focuses on

- **Bit rate:** how much data can be transmitted.
- **Packet loss:** how much data is lost.
- **Bit errors:** how accurately symbols are received.

Semantic communication focuses on

- **Meaning:** what the data represents.
- **Relevance:** which information is useful for the receiver.
- **Task success:** whether the receiver can complete the intended action.

Core shift

Instead of asking:

"Did we transmit all bits correctly?"

Semantic Communications ask:

"Did the receiver get the information needed to achieve the goal?"

What is Semantic Information?

Semantic information is the part of the message that is useful for the receiver's task.

Core idea

Instead of transmitting all raw data, a semantic system tries to extract and send only what matters.

Example:

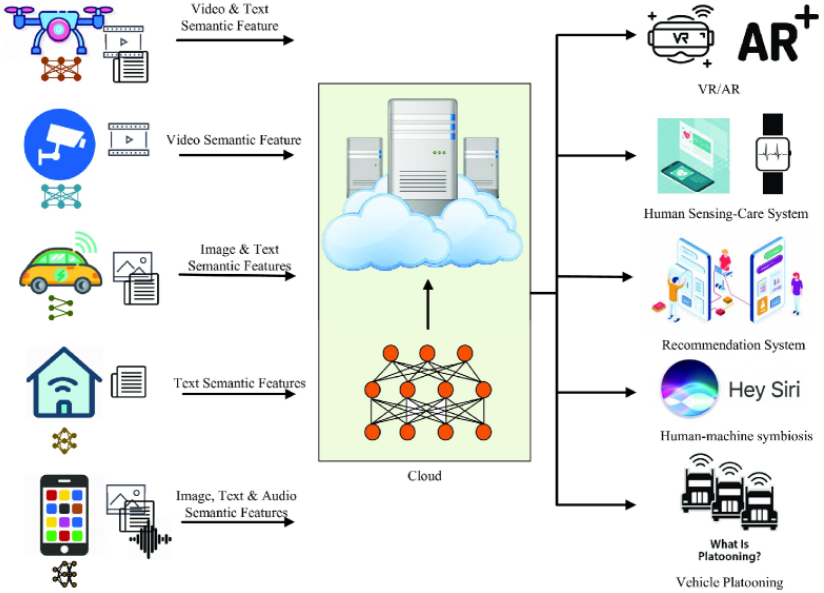
Instead of sending every pixel of a video, the system may send:
"A vehicle is approaching at 50 km/h."

What makes information semantic

- It depends on the **task** of the receiver.
- It uses **context** and shared knowledge.
- It prioritizes information that helps the receiver decide or act.
- It can discard technically correct data that is irrelevant for the goal.

Moving Beyond Shannon

"In the Shannon era, a bit is a bit. In the Semantic era, the value of data depends on its context and the goal of the receiver."



Shared Knowledge and Context

In Semantic Communications, meaning requires shared context between sender and receiver.

Semantic communication depends on

- **Shared knowledge bases**
  Sender and receiver use a common background of concepts, objects, and relations.
- **Common ontologies**
  The system needs a shared way to describe what things are and how they are connected.
- **Synchronized context**
  The receiver must interpret the message using context that is sufficiently aligned with the sender.
- **Reasoning at the receiver**
  The receiver can use its knowledge to reconstruct, complete, or interpret compact messages.

Main risk
A message can be transmitted correctly at the bit level but still be misunderstood if sender and receiver do not share the same context.

Semantic Encoder/Decoder

A semantic communication system does not necessarily transmit raw data. It can transmit compact, meaning-rich features that are useful for the receiver's task.

At the transmitter

The semantic encoder extracts the most relevant information, such as:

- latent features;
- object identities;
- positions and trajectories;
- task-relevant context.

Through the channel

- The system sends a compact semantic representation.
- The objective is not only bit accuracy, but preservation of meaning.

At the receiver

- The semantic decoder reconstructs what is needed for the task.
- The output may not be identical to the original data, but it should preserve the relevant meaning.

Task-Oriented Communication: Measuring What Matters

In task-oriented communication, the objective is not perfect signal reconstruction, but successful task completion.

Traditional question

Did we reconstruct the transmitted signal accurately?

This is measured with classical metrics such as distortion, bit errors, or signal quality.

Task-oriented question

Did the receiver complete the task correctly?

The relevant metric becomes the impact of the received information on the final goal.

Examples of tasks

- Detect an obstacle.
- Guide a robot or drone.
- Classify an object.
- Coordinate vehicles.
- Support a remote operation.

Semantic Noise and Semantic Risk

Semantic errors are different from bit errors: the signal may be technically correct, but the meaning may be wrong.

How semantic errors appear

- A message can be **bit-correct but misunderstood**.
- A message can be **incomplete but still useful** for the task.
- A decoder can **reconstruct missing details incorrectly**.
- An attacker can manipulate the **meaning** without necessarily corrupting the packet.

Why this matters

- Reliability must protect the intended meaning, not only the transmitted bits.
- Security must consider attacks on interpretation and context.
- Trust becomes critical when semantic decoders use AI to reconstruct information.

Use Case: Autonomous V2X

In Semantic V2X, vehicles do not need to exchange all raw sensor data. They need to exchange the information that is relevant for safety and coordination.

What vehicles may share

- **Intentions**
  "I am braking", "I am turning", "I am yielding".
- **Hazards**
  Obstacles, emergency maneuvers, road events, or dangerous conditions.
- **Trajectories**
  Predicted movement of vehicles, pedestrians, or cyclists.
- **Cooperative perception**
  Information detected by one vehicle or roadside unit and shared with others.

Why semantic V2X matters

- Small semantic messages can be more useful than large raw sensor streams.
- Safety-critical information can be prioritized over less relevant data.
- The network can support faster and more coordinated decisions.

Large Models and Semantic Communication

Large models may become part of future semantic communication systems by helping devices extract, compress, and reconstruct meaning.

Possible roles of large models

- **Meaning extraction**
  Transform text, images, audio, or sensor data into compact semantic representations.
- **Multimodal understanding**
  Align information coming from different sources, such as cameras, LiDAR, speech, and haptic signals.
- **Prompt-based communication**
  Transmit a compact description or task instruction, while the receiver reconstructs the needed information locally.
- **Edge intelligence**
  Use smaller models at devices or edge nodes to reduce latency and bandwidth usage.

Main concerns

- Energy and computation cost.
- Trust and hallucinations.
- Privacy and security of semantic representations.

Key Takeaways from Module 4

Future networks will not be evaluated only by how much data they carry, but by how intelligently they support applications and services.

Main messages

- **From throughput to utility**
  Success is not only measured in Gbps, but in whether the service goal is achieved.
- **From passive connectivity to aware networks**
  Future networks will use context, AI, and real-time data to adapt their behavior.
- **From raw data to meaningful information**
  Semantic communication focuses on transmitting what is relevant for the task, not everything.
- **From communication to intelligence support**
  Networks will increasingly support decision-making, automation, and intelligent applications.

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

HR EXCELLENCE IN RESEARCH

04: Emerging Themes

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi
Dipartimento Ingegneria dell'Informazione
daniele.tarchi@unifi.it